Assiut National University

Software Requirements Specification (SRS)

Graduation Project Management System (GPMS)

Faculty of Computers and Artificial intelligence
Department of Software Engineering
Software Engineering Project 2025 / 2026

Feb 9 , 2026

Assiut National University

Faculty Of Computers & Artificial Intelligence

Graduation Project Management System (GPMS)

Software Requirements Specification

(SRS)

Software Engineering Project 2025 / 2026

1. Youssef Mohamed Nasr ElDeen	**2072022214**
2. Omar Abdelrahman Mostafa	**2072022111**
3. Mohamed Osama Mohamed	**2072022137**
4. Abdullah Faisal Shawky	**2072022108**
5. Abdelrahman Naser Fathy	**2072022096**
6. Abdelrahman Hussein Mohamed	**2072022091**
7. Abdelrahman Yasser Mahmoud	**2072022097**

Table of Contents

Table of Contents 2
1. Introduction 4
  1.1 Purpose 4
  1.2 Document Conventions 4
  1.3 Intended Audience and Reading Suggestions 4
  1.4 Product Scope 5
2. Overall Description 6
  2.1 Product Perspective 6
  2.2 Product Functions 6
  2.3 User Classes and Characteristics 7
  2.4 Operating Environment 8
  2.5 Design and Implementation Constraints 8
  2.6 User Documentation 9
  2.7 Assumptions and Dependencies 9
3. External Interface Requirements 10
  3.1 User Interfaces 10
  3.2 Hardware Interfaces 14
  3.3 Software Interfaces 14
  3.4 Communications Interfaces 14
4. System Features 15
  4.0 Enhanced Entity-Relationship Diagram (EERD) 15
  4.1 Student Registration and Email Verification 16
    4.1.1 Description and Priority 16
    4.1.2 Stimulus/Response Sequences 16
    4.1.3 Functional Requirements 16
  4.2 Authentication, Logout, and Role-Based Permissions 17
    4.2.1 Description and Priority 17
    4.2.2 Stimulus/Response Sequences 17
    4.2.3 Functional Requirements 17
  4.3 Admin Creation of Doctor/TA/Admin Accounts 18
5. Other Nonfunctional Requirements 43

5.1 Performance Requirements 43
5.2 Safety Requirements 43
5.3 Security Requirements 43
5.4 Software Quality Attributes 44
5.5 Business Rules 45
6. Other Requirements 46
Appendix A: Glossary 47
Appendix B: Analysis Models 49
1. System Architecture Model (PERN Architecture) 49
2. Use Case Model (UC-01 to UC-24) 49
3. Data Model / Key Entities (Minimum Set) 49
4. Workflow / State Models 49
Appendix C: To Be Determined (TBD) 50

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Graduation Project Management System (GPMS). The document is intended to be used by stakeholders including administrators, students, team leaders, doctors/supervisors, teaching assistants (TAs), and the development team.

This SRS follows the IEEE Software Requirements Specification Template

1.2 Document Conventions

"**Shall**" indicates a mandatory requirement.

"**Should**" indicates a recommended requirement.

"**Could**" indicates an optional requirement.

Feature priority mapping:

- **Must → High**
- **Should → Medium**
- **Could → Low**

Functional requirements are uniquely labeled as REQ-<category>-<number> and written in the form: "The system shall …"

UI figures in Section 3.1 are referenced in Section 4 using figure numbers (e.g., Figure 3-2)

1.3 Intended Audience and Reading Suggestions

Faculty/Project Advisor/Doctor/TA representatives: Review Sections 2, 4, and 5 for workflow and policy expectations.

Administrators: Review Sections 4 (admin-related features), 5 (security/audit/configuration), and 6.

Developers/Testers: Review all sections, especially Section 4 (system features) and Section 5 (measurable non-functional constraints).

Documentation/Support: Review Sections 2.6 and Section 3 (interfaces), plus Appendices.

1.4 Product Scope

GPMS is a web-based system intended to support a graduation cycle by enabling:

- Student registration and verification.
- Team formation using join codes and public-team recruitment.
- Doctor/TA directory browsing and request workflows with deadlines and capacity rules.
- Task management including dependency tracking and TA approval for finished tasks.
- Weekly progress auto-generation and optional AI-assisted rewrite (short text only).
- Meetings with external links, minutes, and topic assignments.
- Team group chat.
- File upload/download with quotas and access rules.
- GitHub OAuth integration for linking a public repository and viewing commits/branches.
- Gamification (XP rules and anti-cheating controls), plus reward store items and coins as described in supporting documents.
- Notifications via email and in-app notifications.
- Optional AI chatbot guidance and AI recommendations with transparency and file-blocking constraints.

2. Overall Description

2.1 Product Perspective

GPMS is a new web application designed to support one graduation cycle. It provides role-based dashboards and workflows for students, team leaders, doctors/supervisors, teaching assistants, and administrators.

The system integrates with external services/interfaces described in the requirements sources, including:

- Email delivery for verification, invites, and notifications.
- GitHub OAuth and GitHub API for repository linking and read-only viewing.
- External meeting links (stored as URLs for tools chosen by users).
- An external AI API provider for optional AI features (short text only).

2.2 Product Functions

Major product functions include:

- Student registration, verification, login/logout.
- Admin creation of doctor/TA/admin accounts.
- Role-based access control.
- Team creation with join code generation.
- Team joining via join code.
- Public/private team visibility.
- Recruitment using proficiency labels and leader approval.
- Doctor/TA profile management (conditions, availability, max teams).
- Doctor/TA directory browsing/searching.
- Doctor/TA request submission with project idea summary.
- Doctor/TA accept/decline workflow (optional decision reason).
- Team activation only when both doctor and TA are assigned.
- Deadline enforcement for requests and admin manual handling.
- Access gating of execution modules until team is active.
- Task management with required fields, limited status values, dependencies, and TA approval flow.

- Weekly progress auto-generation from task data and approvals, with team review/submission.
- Optional AI rewrite of weekly progress (short text only).
- Meeting scheduling with external links, minutes, topics, and topic assignment.
- Team group chat (single chat per active team including assigned doctor and TA).
- File upload/download with quotas and access rules.
- GitHub OAuth connection and public repository linking; read-only branch/commit viewing.
- Gamification XP rules and anti-cheating controls; reward store items and coin purchases where described.
- Notifications via email and in-app notification bar.
- AI chatbot guidance and AI doctor/TA recommendation (advisory only), with mandatory transparency and file-blocking.

2.3 User Classes and Characteristics

Student: Registers, verifies email, joins or forms a team, participates in tasks/meetings/chat/files when team is active.

Team Leader (Student with leader permissions): Creates team, manages recruitment approvals, submits doctor/TA requests, manages tasks and progress submission.

Doctor/Supervisor: Maintains profile (conditions, max teams), reviews and decides doctor requests, participates in team chat, meetings, and progress oversight.

Teaching Assistant (TA): Maintains profile (conditions, max teams), reviews and decides TA requests, approves/rejects finished tasks, participates in team chat/meetings.

Administrator: Creates staff/admin accounts, configures deadlines, manually handles unassigned teams after deadline, and has support access as specified.

2.4 Operating Environment

- **Client:** Web browsers (role-based dashboards; responsive UI expected).
- **Frontend:** React/Next.js (as required by PERN checklist and backend documentation context).
- **Backend:** Node.js + Express.js.
- **Database:** PostgreSQL.
- **Deployment:** Cloud hosting environment (general requirement), with containerized deployment described in backend documentation.
- **Integrations:** Email via SMTP-capable provider; GitHub OAuth and GitHub API; external AI API provider; external meeting links.

2.5 Design and Implementation Constraints

- The system shall be implemented using the **PERN** stack: PostgreSQL, Express.js, React/Next.js, Node.js.
- The system shall provide a web-only MVP as stated in constraints.
- The system shall support one graduation cycle scope as stated in constraints.
- GitHub integration shall be limited to public repositories and read-only viewing for commits and branches.
- AI features shall be optional, transparent, send short text only, and must block file sending to the AI service.
- Requests for doctor/TA shall be blocked after the global deadline, with admin manual handling available.

2.6 User Documentation

User documentation includes:

- END_USER_MANUAL.pdf (end-user manual).
- ADMIN_GUIDE.pdf (administrator guide).

Developer documentation includes:

- BACKEND_COMPLETE_DOCUMENTATION_PERN.pdf.

2.7 Assumptions and Dependencies

- The system depends on an email service to deliver verification, invite, request decision, and notification emails.
- The system depends on GitHub OAuth and GitHub API availability for repository integration features.
- Meetings depend on external meeting links chosen by users (stored as URLs).
- AI features depend on an external AI API provider and require explicit user confirmation before sending text.
- The system assumes one graduation cycle and supports retention/archival considerations as described (see Section 6 and Appendix C for open policy items).

3. External Interface Requirements

3.1 User Interfaces

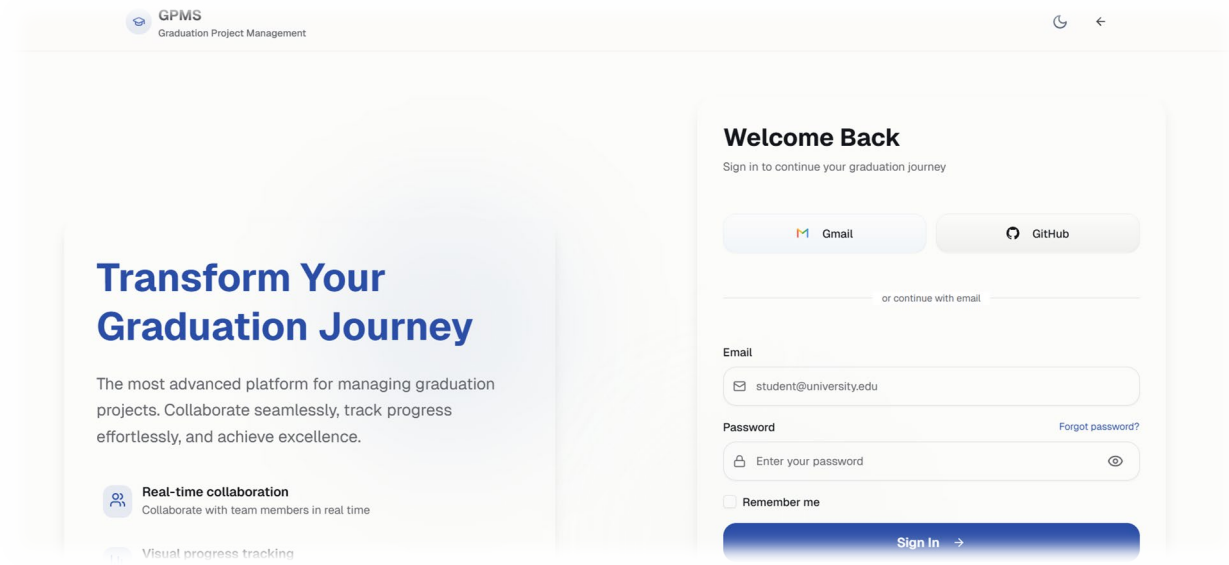

Figure 3-1: Login Screen

The login screen shall provide email/password authentication for all roles. It shall display errors for invalid credentials and shall block access for unverified or inactive accounts. Successful login shall redirect users to a role-based dashboard.

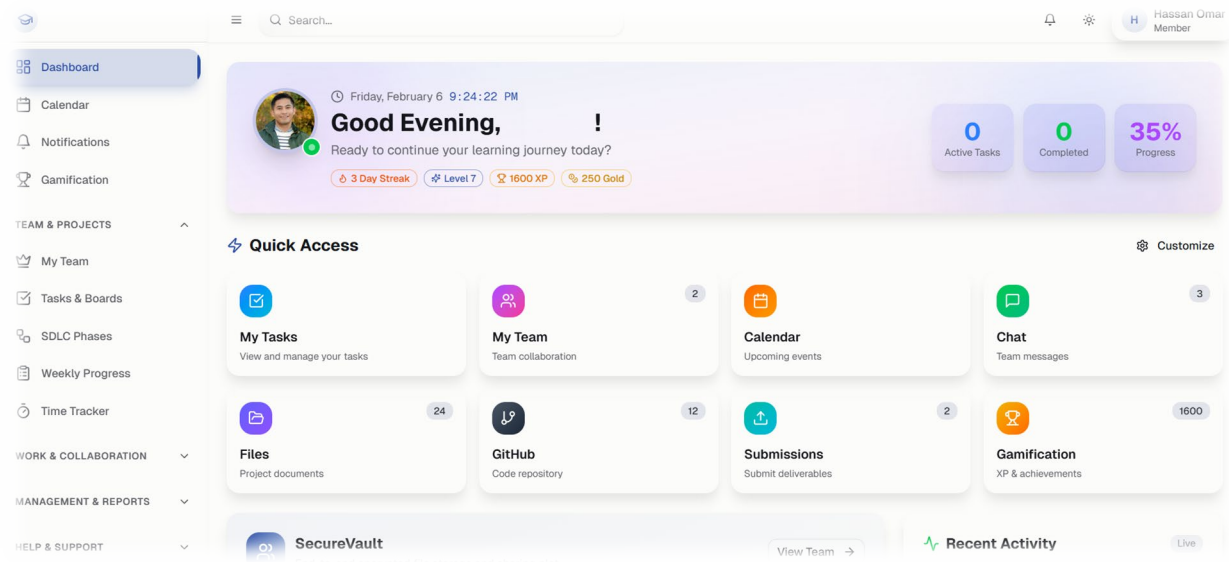

Figure 3-2: Student Dashboard

The student dashboard shall present role-relevant summaries such as team status (e.g., not active vs active), notifications, and high-level indicators relevant to the student's participation in team tasks, meetings, and progress once the team is active.

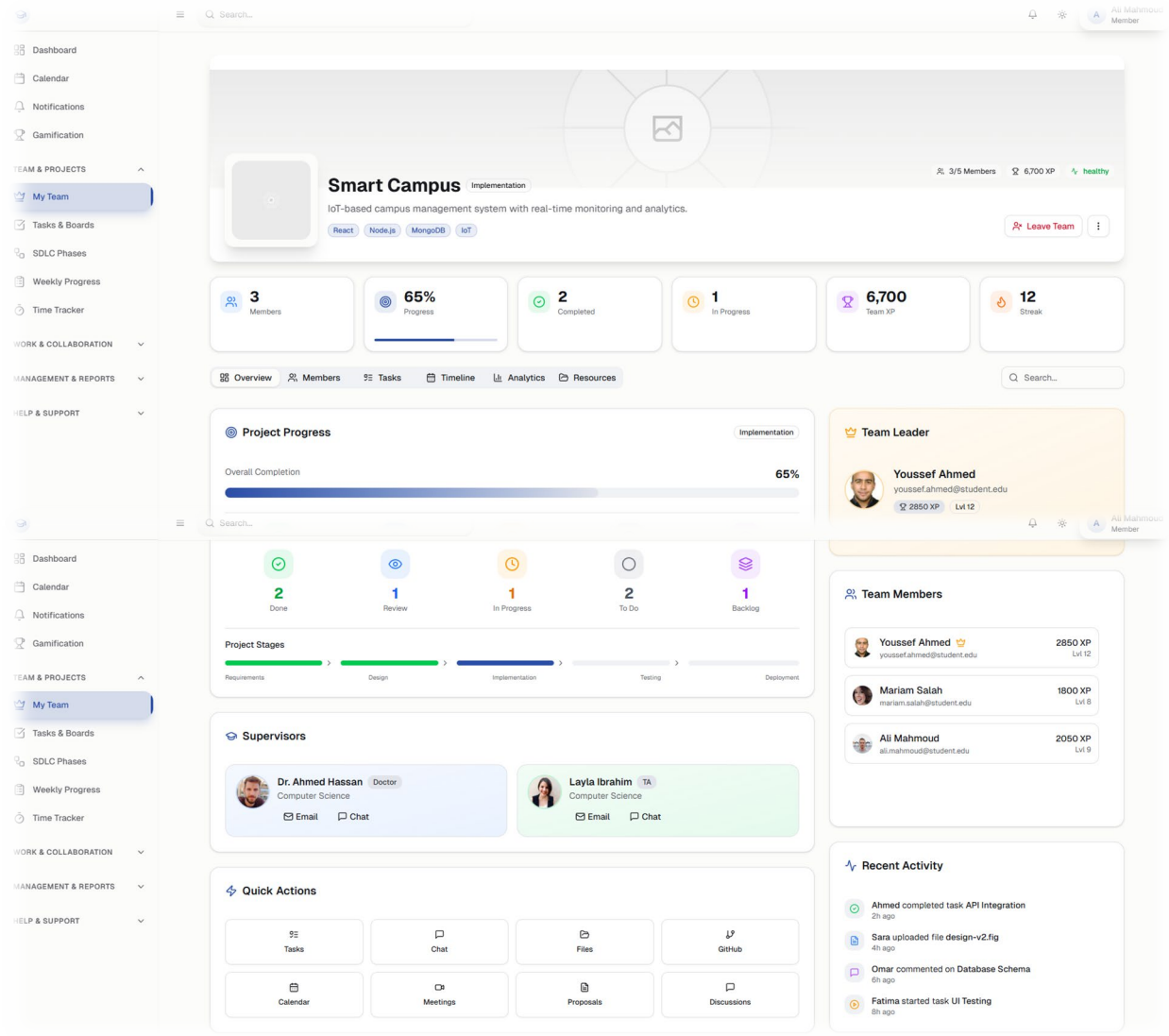

Figure 3-3: Team Dashboard Screen

The team dashboard shall present team information (team identity and membership context), team activation status, and access to team modules that are available only when the team is active (tasks, meetings, chat, files, repository view), consistent with the access gating requirement.

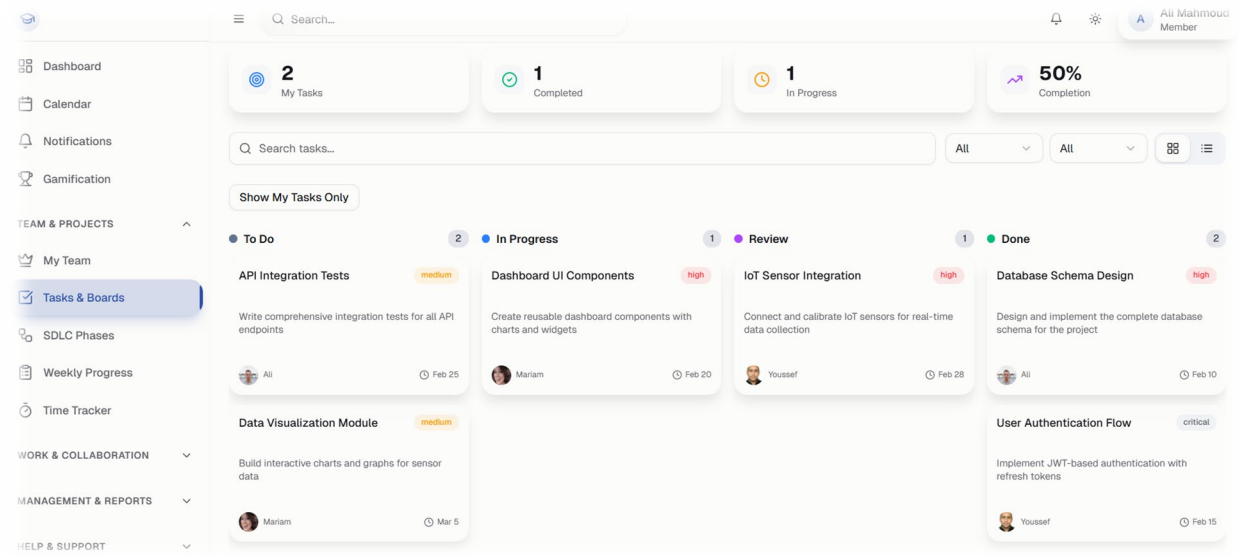

Figure 3-4: Task Board / Gantt View

The task board shall display tasks with required fields and constrained status values. A timeline view (Gantt-like) shall be provided as a simple visualization if enabled, without advanced project-management enforcement.

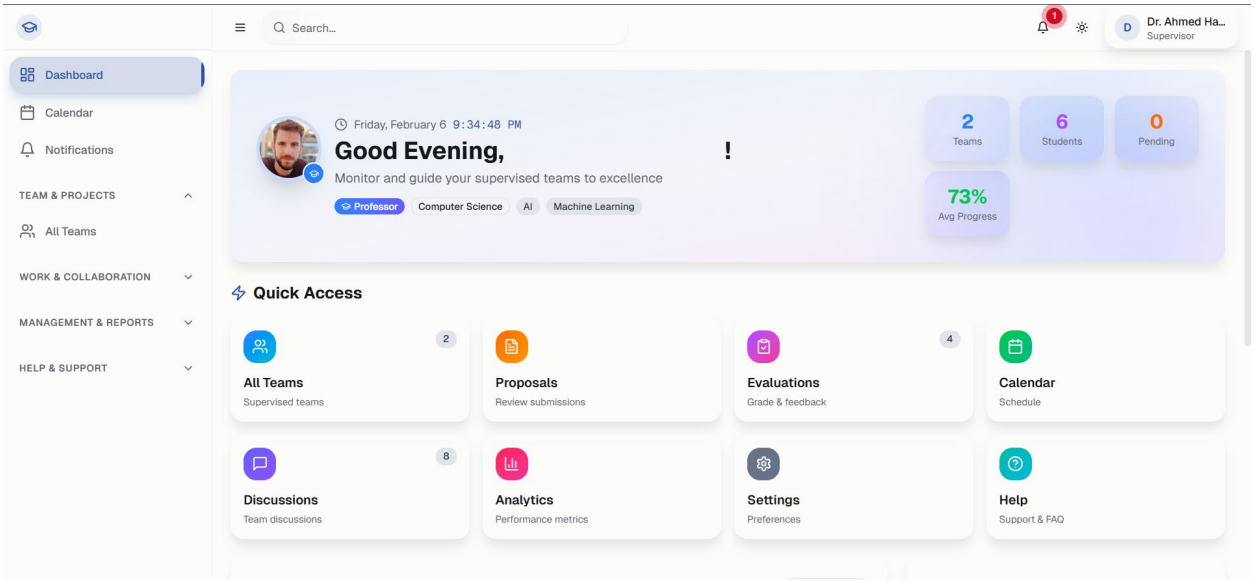

Figure 3-5: Supervisor Dashboard

The supervisor dashboard shall allow a doctor/supervisor to view requests, view assigned team context, and participate in team oversight features (meetings, chat, progress visibility) consistent with role permissions and team activation gating.

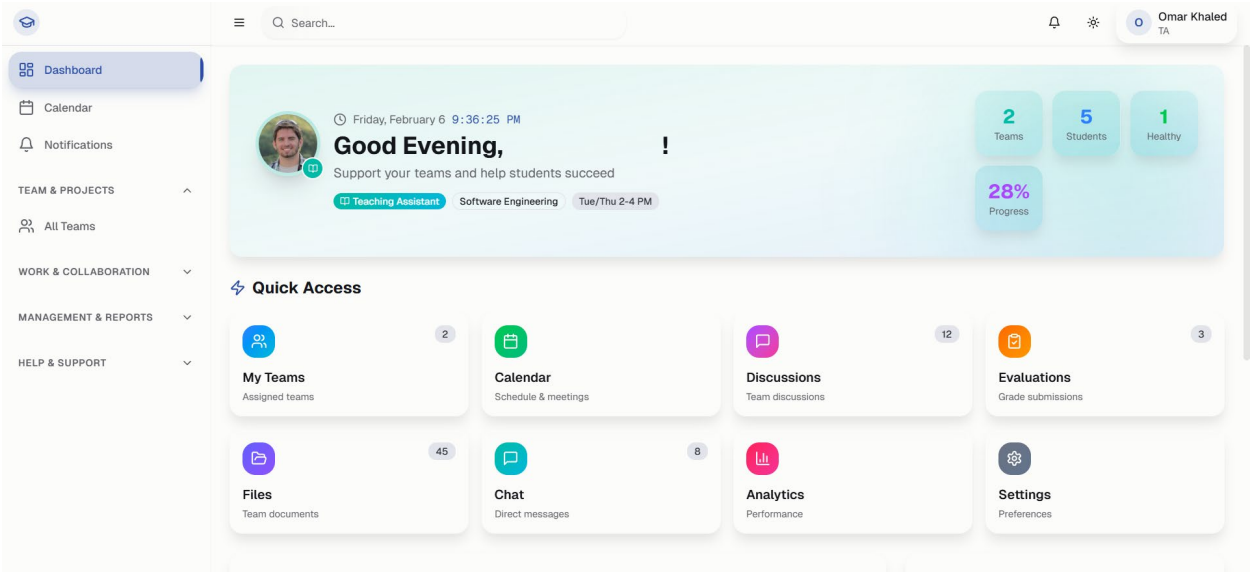

Figure 3-6: TA Dashboard

The TA dashboard shall allow a TA to view requests, manage task approval for finished tasks, and participate in team oversight features (meetings, chat, progress visibility) consistent with role permissions and team activation gating.

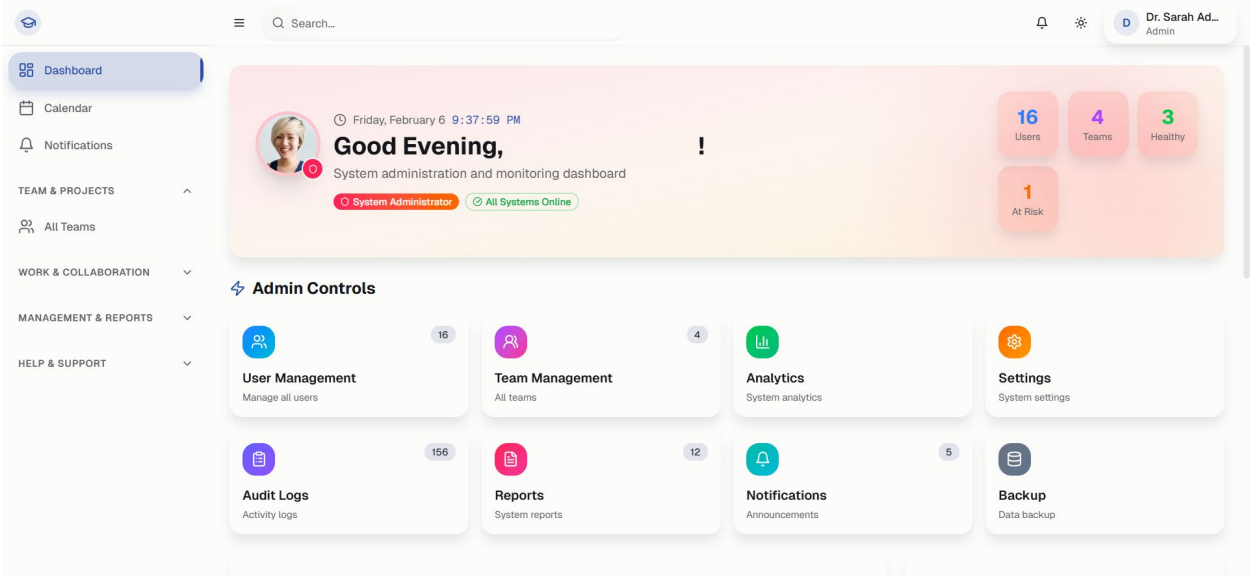

Figure 3-7: Admin Dashboard

The admin dashboard shall provide administrative visibility and control operations described in the requirements sources, including account creation for doctor/TA/admin, deadline configuration and manual handling after deadline, and support access as applicable.

3.2 Hardware Interfaces

No specialized hardware interfaces are required. GPMS is accessed through standard web-capable devices.

3.3 Software Interfaces

- **Database:** PostgreSQL for persistent storage of system entities and records.
- **GitHub:** OAuth authorization and GitHub API access for public repository linking and read-only data retrieval (branches and commits).
- **Email:** SMTP-compatible email delivery to send verification emails, invites, decision notifications, meeting reminders, and general notifications.
- **AI Provider:** External AI API provider used only for short text inputs for chatbot/rewrite/recommendations, with transparency and file-blocking requirements.
- **External Meetings:** Meeting links stored as URLs (e.g., video meeting platforms chosen by users).

3.4 Communications Interfaces

- **Web API:** HTTPS with REST/JSON interfaces.
- **Real-time communication:** WebSocket-based communication for real-time features where applicable (as described in backend documentation context).
- **Email transport:** SMTP communication with configured email service.
- **OAuth:** Standard OAuth-based authorization flow for GitHub connection.

4. System Features

4.0 Enhanced Entity-Relationship Diagram (EERD)

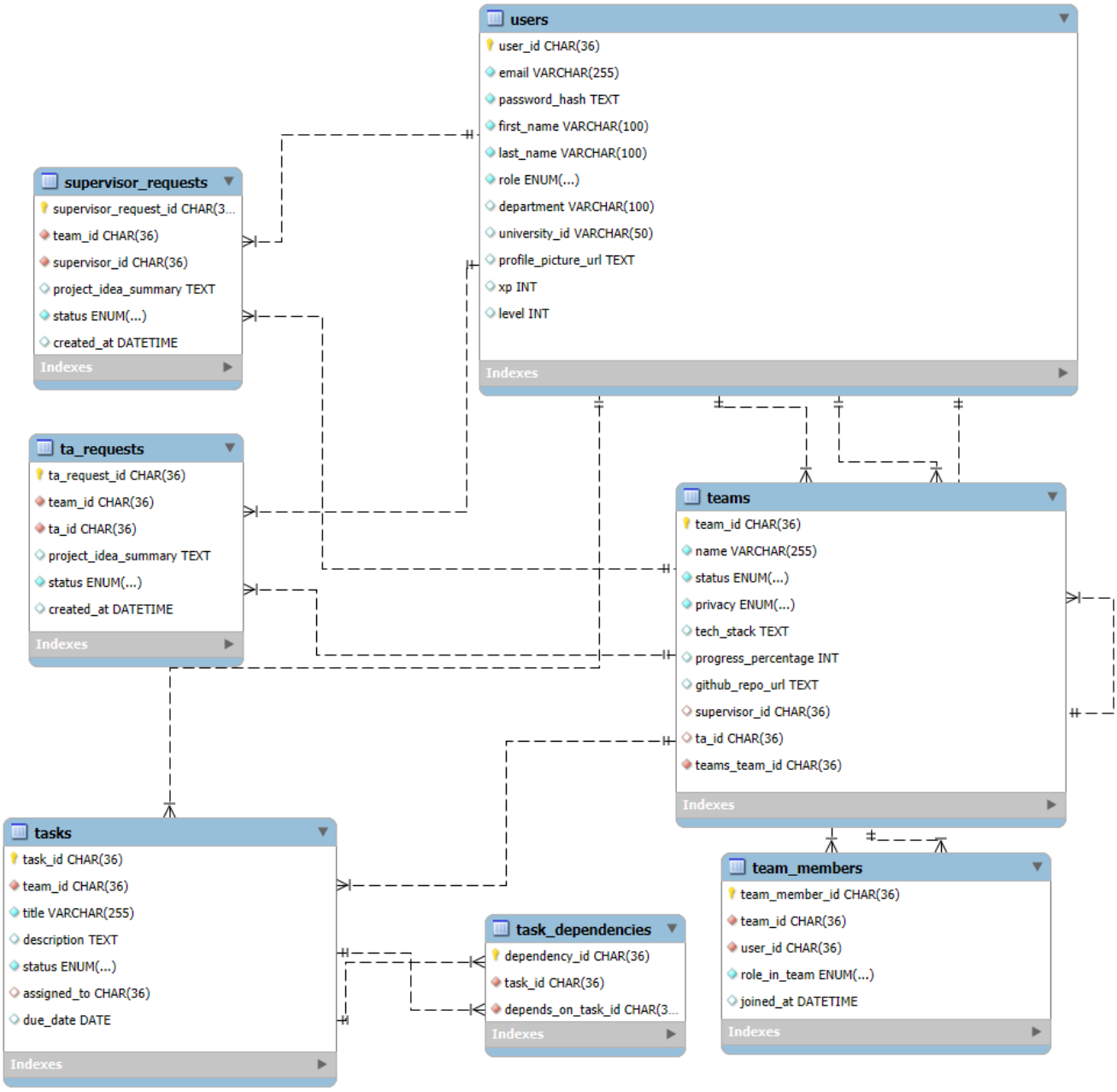

Figure 3-8: EERD

4.1 Student Registration and Email Verification

4.1.1 Description and Priority

This feature supports student self-registration and mandatory email verification before system usage.

Priority: High

4.1.2 Stimulus/Response Sequences

Stimulus: A student opens the registration page and submits required information.

Response: The system validates uniqueness and formats, creates an unverified student account, and sends a verification message.

Stimulus: The student verifies using a link or OTP.

Response: The system marks the account verified and allows login.

Stimulus: The verification token is invalid/expired.

Response: The system rejects verification and allows requesting a new verification email.

4.1.3 Functional Requirements

REQ-REG-01: The system shall allow a student to self-register using university email, name, university ID, and department.

REQ-REG-02: The system shall validate that the registration email is unique; if a duplicate email exists, the system shall reject registration and display an error.

REQ-REG-03: The system shall validate that the university ID is unique; if a duplicate ID exists, the system shall reject registration and display an error.

REQ-REG-04: The system shall validate email format and university domain; if invalid, the system shall reject registration and display an error.

REQ-REG-05: The system shall create the student account with the Student role in an unverified state after successful validation.

REQ-REG-06: The system shall send an email verification message to the registered email address after successful registration.

REQ-REG-07: The system shall support verification by link and/or OTP as provided in the verification process description.

REQ-REG-08: The system shall prevent unverified students from accessing the system until verification is completed.

REQ-REG-09: The system shall allow a student to request a new verification email when the previous verification token is invalid or expired.

4.2 Authentication, Logout, and Role-Based Permissions

4.2.1 Description and Priority

This feature provides login/logout and enforces role-based access control for system actions and protected pages.

Priority: High

4.2.2 Stimulus/Response Sequences

Stimulus: A user enters credentials on the login screen (Figure 3-1).

Response: The system authenticates and routes the user to a role-based dashboard.

Stimulus: A user attempts to access a protected page without authentication.

Response: The system blocks access and requires login.

Stimulus: A logged-in user selects logout.

Response: The system ends the session and blocks further access until login.

4.2.3 Functional Requirements

REQ-AUTH-01: The system shall allow users to log in using email and password.

REQ-AUTH-02: The system shall block login for unverified or inactive accounts and display an appropriate error message.

REQ-AUTH-03: The system shall redirect authenticated users to the dashboard for their role upon successful login.

REQ-AUTH-04: The system shall terminate the authenticated session when the user logs out.

REQ-AUTH-05: The system shall require authentication for access to protected pages and protected backend APIs.

REQ-AUTH-06: The system shall enforce permissions based on user role and leader/member permissions for every protected action.

REQ-AUTH-07: The system shall deny unauthorized action attempts and shall optionally log these attempts as described by security/audit expectations.

4.3 Admin Creation of Doctor/TA/Admin Accounts

4.3.1 Description and Priority

This feature enables administrators to create doctor, TA, and admin accounts and provide credentials.

Priority: High

4.3.2 Stimulus/Response Sequences

Stimulus: Admin creates a new staff/admin account from the admin interface (Figure 3-7).

Response: The system validates uniqueness and creates the account with the selected role and temporary password.

Stimulus: Admin attempts to create a user with a duplicate email.

Response: The system rejects the request and displays an error.

4.3.3 Functional Requirements

REQ-ADM-ACC-01: The system shall allow an authenticated administrator to create doctor, TA, and administrator accounts.

REQ-ADM-ACC-02: The system shall require the administrator to specify user data and role when creating such accounts.

REQ-ADM-ACC-03: The system shall reject account creation if the email is already in use.

REQ-ADM-ACC-04: The system shall generate a temporary password for created accounts and store it securely.

REQ-ADM-ACC-05: The system shall create the account with the correct role and enable login using the provided credentials.

4.4 Team Creation and Join Code Generation

4.4.1 Description and Priority

This feature allows a student leader to create a team and generates a unique join code.

Priority: High

4.4.2 Stimulus/Response Sequences

Stimulus: A student initiates team creation and submits team name and privacy (Figure 3-3).

Response: The system creates the team, generates a join code, and assigns the creator as leader.

Stimulus: A student who is already in a team attempts to create another team.

Response: The system blocks the action and displays an error.

4.4.3 Functional Requirements

REQ-TEAM-CRT-01: The system shall allow a student who is not already in a team to create a team.

REQ-TEAM-CRT-02: The system shall require the team leader to provide a team name and privacy setting during creation.

REQ-TEAM-CRT-03: The system shall generate a unique join code when a team is created.

REQ-TEAM-CRT-04: The system shall assign the creating student as the team leader upon successful creation.

REQ-TEAM-CRT-05: The system shall block team creation if the student is already a member of another team.

REQ-TEAM-CRT-06: The system shall block team creation if the team name is missing.

4.5 Team Size Enforcement (3–8)

4.5.1 Description and Priority

This feature enforces team member count rules.

Priority: High

4.5.2 Stimulus/Response Sequences

Stimulus: A join or approval action would increase the team size beyond the maximum.

Response: The system blocks the action and displays an error.

4.5.3 Functional Requirements

REQ-TEAM-SIZE-01: The system shall enforce a team member count between 3 and 8 as a defined rule.

REQ-TEAM-SIZE-02: The system shall block any join or approval action that would cause a team to exceed 8 members.

REQ-TEAM-SIZE-03: The system shall display an error message when a join/approval is blocked due to team capacity.

4.6 Join Team Using Join Code

4.6.1 Description and Priority

This feature enables students to join a team using a join code, subject to eligibility and capacity.

Priority: High

4.6.2 Stimulus/Response Sequences

Stimulus: A student enters a join code (Figure 3-3).

Response: The system displays team information and proceeds according to team privacy rules.

Stimulus: A student enters an invalid code.

Response: The system displays an error.

Stimulus: A student already in a team attempts to join another team.

Response: The system blocks the action.

4.6.3 Functional Requirements

REQ-TEAM-JOIN-01: The system shall allow a verified student who is not in another team to enter a team join code to join a team.

REQ-TEAM-JOIN-02: The system shall display team information associated with a valid join code before completion of the join action.

REQ-TEAM-JOIN-03: The system shall reject invalid join codes and display an error message.

REQ-TEAM-JOIN-04: The system shall block join attempts if the team is full.

REQ-TEAM-JOIN-05: The system shall block join attempts if the student is already a member of another team.

REQ-TEAM-JOIN-06: The system shall apply join/request behavior based on the team's privacy rules as described.

4.7 Invite Members Using (Name, Email, Join Code)

4.7.1 Description and Priority

This feature enables team leaders to invite members by email, including join code and instructions.

Priority: High

4.7.2 Stimulus/Response Sequences

Stimulus: Leader enters invitee name and email (Figure 3-3).

Response: System sends invitation email including join code and instructions.

Stimulus: Invalid email format/domain is provided.

Response: System rejects and displays error.

4.7.3 Functional Requirements

REQ-TEAM-INV-01: The system shall allow a team leader to invite members by entering invitee name and email.

REQ-TEAM-INV-02: The system shall validate invitee email format and domain and reject invalid emails.

REQ-TEAM-INV-03: The system shall prevent inviting new members when the team is full.

REQ-TEAM-INV-04: The system shall send an invitation email that includes the team join code and join instructions.

REQ-TEAM-INV-05: The system shall support optional invite status tracking and/or resend capability as indicated.

4.8 Public/Private Team Visibility and Recruitment by Proficiency

4.8.1 Description and Priority

This feature supports public/private visibility and recruitment based on proficiency labels with leader approval.

Priority: High

4.8.2 Stimulus/Response Sequences

Stimulus: Student without a team browses the public teams list.

Response: System shows public teams and hides private teams.

Stimulus: Leader posts needed proficiency labels and a student applies.

Response: System records application and routes to leader for approval/decline.

4.8.3 Functional Requirements

REQ-RECRUIT-01: The system shall support public and private teams and apply visibility rules accordingly.

REQ-RECRUIT-02: The system shall display public teams in a public teams list to students, particularly students without a team.

REQ-RECRUIT-03: The system shall not display private teams in the public teams list.

REQ-RECRUIT-04: The system shall allow leaders of public teams to request needed proficiency labels for recruitment.

REQ-RECRUIT-05: The system shall allow students not in a team to apply to a public team.

REQ-RECRUIT-06: The system shall allow the team leader to approve or decline applicants without administrator approval.

REQ-RECRUIT-07: The system shall block approvals when a team is full.

REQ-RECRUIT-08: The system shall notify declined applicants as specified.

4.9 Student Proficiency Labels

4.9.1 Description and Priority

This feature allows students to label their proficiencies to support recruitment and visibility.

Priority: Medium

4.9.2 Stimulus/Response Sequences

Stimulus: Student selects proficiency labels in their profile.

Response: System saves and displays labels in relevant contexts (profiles and team dashboard).

4.9.3 Functional Requirements

REQ-PROF-01: The system shall allow students to add proficiency labels to their profile.

REQ-PROF-02: The system shall store selected proficiency labels for each student profile.

REQ-PROF-03: The system shall display proficiency labels on the student profile and team dashboard (Figure 3-3).

4.10 Doctor/TA Conditions, Availability, and Max Teams

4.10.1 Description and Priority

Doctors and TAs define conditions, availability (optional time slots), and maximum team capacity.

Priority: High

4.10.2 Stimulus/Response Sequences

Stimulus: Doctor/TA edits conditions and max teams (Figure 3-5 / Figure 3-6).

Response: System validates and publishes updated profile information.

Stimulus: Invalid max teams value is entered.

Response: System rejects and displays an error.

4.10.3 Functional Requirements

REQ-STAFF-01: The system shall allow doctors and TAs to define and update conditions information.

REQ-STAFF-02: The system shall allow doctors and TAs to set a maximum number of teams they can supervise/support.

REQ-STAFF-03: The system shall allow doctors and TAs to optionally add availability time slots.

REQ-STAFF-04: The system shall validate max teams inputs and reject invalid values.

REQ-STAFF-05: The system shall display conditions, availability, and max teams in the doctor/TA directory.

4.11 Browse Doctor/TA Directory and Capacity Blocking

4.11.1 Description and Priority

Students browse/search the doctor/TA directory and requests are blocked when capacity is reached.

Priority: High

4.11.2 Stimulus/Response Sequences

Stimulus: Student searches directory.

Response: System displays results with conditions, availability, and max teams.

Stimulus: Leader attempts to request a doctor/TA who is full.

Response: System blocks the request submission and displays an error.

4.11.3 Functional Requirements

REQ-DIR-01: The system shall allow students to browse and search doctors and TAs and view their profiles.

REQ-DIR-02: The system shall display conditions, availability, and max teams for each doctor/TA profile.

REQ-DIR-03: The system shall block leader requests to a doctor/TA when the doctor/TA has reached max teams.

REQ-DIR-04: The system shall determine "full" status based on the number of assigned teams relative to max teams.

REQ-DIR-05: The system shall allow administrator manual assignment to override capacity blocking after the deadline as specified.

4.12 Doctor Request With Project Idea Summary and Accept/Decline

4.12.1 Description and Priority

Team leaders submit doctor requests with a required project idea summary; doctors accept/decline with optional reason.

Priority: High

4.12.2 Stimulus/Response Sequences

Stimulus: Leader submits doctor request with summary (Figure 3-3).

Response: System creates pending request and notifies the doctor (Figure 3-5).

Stimulus: Doctor accepts or declines.

Response: System assigns doctor (if accepted) or notifies leader with decline result.

4.12.3 Functional Requirements

REQ-DR-REQ-01: The system shall allow a team leader to submit a doctor request that includes a project idea summary.

REQ-DR-REQ-02: The system shall reject doctor requests missing the project idea summary.

REQ-DR-REQ-03: The system shall block doctor request submission after the global deadline.

REQ-DR-REQ-04: The system shall create a doctor request with status Pending upon successful submission.

REQ-DR-REQ-05: The system shall notify the selected doctor when a pending request is submitted.

REQ-DR-DEC-01: The system shall allow a doctor to accept or decline pending doctor requests.

REQ-DR-DEC-02: The system shall allow the doctor to optionally provide a decision reason when declining.

REQ-DR-DEC-03: The system shall assign the doctor to the team when the doctor accepts the request.

REQ-DR-DEC-04: The system shall notify the team leader when the doctor declines the request.

REQ-DR-DEC-05: The system shall block acceptance if the doctor becomes full before accepting and shall notify the doctor accordingly.

4.13 TA Request With Project Idea Summary and Accept/Decline

4.13.1 Description and Priority

Team leaders submit TA requests with a project idea summary; TAs accept/decline with optional reason.

Priority: High

4.13.2 Stimulus/Response Sequences

Stimulus: Leader submits TA request with summary.

Response: System creates pending request and notifies the TA (Figure 3-6).

Stimulus: TA accepts or declines.

Response: System assigns TA (if accepted) or notifies leader with decline result.

4.13.3 Functional Requirements

REQ-TA-REQ-01: The system shall allow a team leader to submit a TA request that includes a project idea summary.

REQ-TA-REQ-02: The system shall block TA request submission after the global deadline.

REQ-TA-REQ-03: The system shall create a TA request with status Pending upon successful submission.

REQ-TA-REQ-04: The system shall notify the selected TA when a pending request is submitted.

REQ-TA-DEC-01: The system shall allow a TA to accept or decline pending TA requests.

REQ-TA-DEC-02: The system shall allow the TA to optionally provide a decision reason when declining.

REQ-TA-DEC-03: The system shall assign the TA to the team when the TA accepts the request.

REQ-TA-DEC-04: The system shall notify the team leader when the TA declines the request.

4.14 Team Active Status and Access Gating Before Assignment

4.14.1 Description and Priority

Teams become active only after both doctor and TA are assigned, and execution modules are gated until activation.

Priority: High

4.14.2 Stimulus/Response Sequences

Stimulus: Doctor assignment and TA assignment both exist for a team.

Response: System marks the team active and unlocks modules.

Stimulus: Team attempts to access tasks/chat/meetings/files/repo while not active.

Response: System blocks access and displays a waiting message.

4.14.3 Functional Requirements

REQ-ACT-01: The system shall mark a team as Active only when both a doctor and a TA are assigned.

REQ-ACT-02: The system shall restrict tasks, chat, meetings, files, and repository views to active teams only.

REQ-ACT-03: The system shall display a "Waiting for assignment" message and block execution modules for non-active teams.

REQ-ACT-04: The system shall allow administrators to access team modules for support as an exception.

4.15 Deadline Enforcement and Admin Manual Handling

4.15.1 Description and Priority

After the global deadline, teams cannot submit new requests; administrators may extend a deadline for a team or manually assign doctor/TA to unassigned teams.

Priority: High

4.15.2 Stimulus/Response Sequences

Stimulus: Leader tries to submit a request after deadline.

Response: System blocks the action.

Stimulus: Admin reviews unassigned teams after deadline.

Response: Admin extends deadline for a team or manually assigns doctor/TA.

4.15.3 Functional Requirements

REQ-DL-01: The system shall allow an administrator to configure a global deadline used for request enforcement.

REQ-DL-02: The system shall block leaders from submitting new doctor/TA requests after the global deadline.

REQ-DL-03: The system shall allow administrators to extend the deadline for a team.

REQ-DL-04: The system shall allow administrators to manually assign a doctor and/or TA to unassigned teams after the deadline.

4.16 Task Management With Required Fields and Status Values

4.16.1 Description and Priority

Active teams manage tasks with required fields and constrained status values. Task views are presented through the task board (Figure 3-4).

Priority: High

4.16.2 Stimulus/Response Sequences

Stimulus: Authorized user creates or edits a task.

Response: System validates required fields and saves the task.

Stimulus: User sets an invalid task status.

Response: System rejects the update.

4.16.3 Functional Requirements

REQ-TASK-01: The system shall allow active teams to create tasks and edit tasks based on role permissions.

REQ-TASK-02: The system shall require tasks to include the required fields identified: status, priority, dependencies, and TA approval state.

REQ-TASK-03: The system shall reject saving a task when required fields are missing.

REQ-TASK-04: The system shall restrict task status values to upcoming, in progress, and finished.

REQ-TASK-05: The system shall reject invalid status values and display an error.

4.17 Task Dependencies

4.17.1 Description and Priority

Tasks support dependencies on other tasks within the same team and prevent circular dependency relationships.

Priority: High

4.17.2 Stimulus/Response Sequences

Stimulus: Authorized user selects dependency tasks.

Response: System validates same-team dependencies and saves if valid.

Stimulus: Circular dependency is detected.

Response: System blocks the update and displays a warning.

4.17.3 Functional Requirements

REQ-DEP-01: The system shall allow tasks to depend on other tasks in the same team.

REQ-DEP-02: The system shall reject dependencies that reference tasks outside the team.

REQ-DEP-03: The system shall detect circular dependencies and block them with a warning.

REQ-DEP-04: The system shall display task dependencies in task details views.

4.18 TA Approval For Finished Tasks

4.18.1 Description and Priority

When a task becomes finished, it requires TA approval (Approved/Rejected). Rejected tasks return to in progress.

Priority: High

4.18.2 Stimulus/Response Sequences

Stimulus: Student marks a task as finished.

Response: System records finished status and awaits TA review.

Stimulus: TA approves.

Response: System marks TA approval as approved and counts the task as completed for progress and XP.

Stimulus: TA rejects.

Response: System returns task to in progress and marks approval rejected.

4.18.3 Functional Requirements

REQ-APR-01: The system shall require TA approval when a task status becomes finished.

REQ-APR-02: The system shall allow the TA to approve or reject the finished task.

REQ-APR-03: The system shall return the task status to in progress when the TA rejects the task.

REQ-APR-04: The system shall count the task as completed for progress tracking only when the TA approves.

4.19 Simple Timeline (Gantt-like) View

4.19.1 Description and Priority

The system provides a simple timeline view for tasks without advanced enforcement features.

Priority: Medium

4.19.2 Stimulus/Response Sequences

Stimulus: User opens timeline view (Figure 3-4).

Response: System displays tasks along dates; tasks without dates appear as unscheduled.

4.19.3 Functional Requirements

REQ-TLINE-01: The system shall provide a simple timeline view for tasks.

REQ-TLINE-02: The system shall display tasks without dates as unscheduled in the timeline view.

REQ-TLINE-03: The system shall ensure the timeline view reflects the same task data shown in the task board.

4.20 Weekly Progress Auto-Generation

4.20.1 Description and Priority

Weekly progress drafts are auto-generated from task status and TA approvals, then reviewed and submitted by the team.

Priority: High

4.20.2 Stimulus/Response Sequences

Stimulus: Weekly cycle triggers progress generation.

Response: System produces an auto-draft that reflects tasks and approvals.

Stimulus: Team reviews and submits weekly progress.

Response: System stores final weekly progress record.

4.20.3 Functional Requirements

REQ-WP-01: The system shall auto-generate weekly progress drafts based on task status and TA approvals.

REQ-WP-02: The system shall include approved completions, ongoing tasks, and pending approvals in the weekly draft.

REQ-WP-03: The system shall generate a draft indicating no tracked work when no tasks exist.

REQ-WP-04: The system shall allow the team to review and submit a weekly progress record derived from the weekly draft.

REQ-WP-05: The system shall store weekly progress records including auto-draft text, final text, submitter identity, and submission timestamp.

4.21 AI Weekly Summary Rewrite

4.21.1 Description and Priority

AI assistance may be used to rewrite weekly progress drafts into clearer summaries, using short text only and requiring user confirmation.

Priority: Medium

4.21.2 Stimulus/Response Sequences

Stimulus: User clicks "Improve with AI".

Response: System displays AI notice, requests consent, sends short text to AI API, and displays labeled AI output.

Stimulus: AI API fails.

Response: System displays error and allows manual editing.

4.21.3 Functional Requirements

REQ-AI-WP-01: The system shall provide an AI-assisted rewrite option for weekly progress drafts using short text only.

REQ-AI-WP-02: The system shall display a notice that text will be sent to an external AI service before executing an AI rewrite.

REQ-AI-WP-03: The system shall require user agreement before sending text to the external AI service.

REQ-AI-WP-04: The system shall label AI-generated output as AI-generated and require the user to confirm/save it.

REQ-AI-WP-05: The system shall not auto-submit weekly progress based on AI output.

REQ-AI-WP-06: The system shall allow users to continue manually if the AI API is unavailable or fails.

4.22 Meeting Scheduling + External Link

4.22.1 Description and Priority

Active teams schedule meetings with date/time and an external meeting link; meeting appears in team/doctor/TA contexts.

Priority: High

4.22.2 Stimulus/Response Sequences

Stimulus: Doctor/TA/Leader creates a meeting.

Response: System validates inputs, stores meeting, and notifies participants.

Stimulus: Invalid time is provided.

Response: System rejects the meeting creation.

4.22.3 Functional Requirements

REQ-MTG-01: The system shall allow active teams to schedule meetings with date/time and an external meeting link.

REQ-MTG-02: The system shall allow meeting creation by doctor, TA, and student leader roles as identified.

REQ-MTG-03: The system shall validate meeting time input and reject invalid values.

REQ-MTG-04: The system shall notify meeting participants when a meeting is scheduled.

REQ-MTG-05: The system shall display meetings for the team and the assigned doctor and TA.

4.23 Meeting Minutes + Topics + Topic Assignment

4.23.1 Description and Priority

Meeting records store minutes and topics, and topics can be assigned to members.

Priority: High

4.23.2 Stimulus/Response Sequences

Stimulus: Authorized user adds meeting minutes and topics.

Response: System stores minutes/topics and displays them within meeting details.

Stimulus: Topic is assigned to a member.

Response: System stores the assignment.

4.23.3 Functional Requirements

REQ-MIN-01: The system shall store meeting minutes for each meeting.

REQ-MIN-02: The system shall store discussed topics for each meeting.

REQ-MIN-03: The system shall allow topics to be assigned to members.

REQ-MIN-04: The system shall display minutes/topics and topic assignments within meeting details.

4.24 Team Group Chat

4.24.1 Description and Priority

Each active team has one group chat including team members and the assigned doctor and TA. No 1:1 messaging is required.

Priority: High

4.24.2 Stimulus/Response Sequences

Stimulus: Authorized participant sends a message.

Response: System stores and displays message to all participants.

Stimulus: Unauthorized user attempts access.

Response: System blocks access.

4.24.3 Functional Requirements

REQ-CHAT-01: The system shall provide one group chat per active team including team members, assigned doctor, and assigned TA.

REQ-CHAT-02: The system shall allow authorized participants to send and read messages in the team chat.

REQ-CHAT-03: The system shall store chat messages with sender identity and timestamp.

REQ-CHAT-04: The system shall block unauthorized users from accessing or sending messages in the chat.

REQ-CHAT-05: The system shall not require 1:1 messaging functionality.

4.25 File Upload/Download With Quotas

4.25.1 Description and Priority

Active teams upload/download documents with enforced quotas and restricted access rules.

Priority: High

4.25.2 Stimulus/Response Sequences

Stimulus: Authorized user uploads a file.

Response: System validates type/size/quota, stores file, and lists it.

Stimulus: File violates rules (size/type/quota).

Response: System rejects and displays an error.

4.25.3 Functional Requirements

REQ-FILE-01: The system shall allow authorized users in an active team to upload files.

REQ-FILE-02: The system shall enforce a maximum of 100MB per file.

REQ-FILE-03: The system shall enforce a maximum of 500MB total storage per team.

REQ-FILE-04: The system shall reject uploads that exceed size limits, violate type rules, or exceed team quota and display an error message.

REQ-FILE-05: The system shall list uploaded files for authorized users.

REQ-FILE-06: The system shall allow file download only to team members and the assigned doctor and TA.

REQ-FILE-07: The system shall allow administrators to access files for support purposes as an exception.

4.26 GitHub OAuth Connect + Link Public Repo

4.26.1 Description and Priority

Active teams connect GitHub via OAuth and link a public repository for read-only viewing.

Priority: High

4.26.2 Stimulus/Response Sequences

Stimulus: Leader initiates GitHub connection.

Response: System performs OAuth authorization and stores the connection reference; repo URL is saved.

Stimulus: OAuth is denied.

Response: System shows error and does not connect.

4.26.3 Functional Requirements

REQ-GH-01: The system shall allow active teams to connect GitHub via OAuth.

REQ-GH-02: The system shall require the linked repository to be public.

REQ-GH-03: The system shall store the repository URL and connection reference after successful OAuth authorization.

REQ-GH-04: The system shall display an error and not connect GitHub when OAuth authorization is denied.

REQ-GH-05: The system shall display the linked repository information on the team dashboard (Figure 3-3).

4.27 View Branches and Commits (Read-only)

4.27.1 Description and Priority

The system displays branches and commits from the linked public repository and handles failures gracefully.

Priority: High

4.27.2 Stimulus/Response Sequences

Stimulus: User opens repository view.

Response: System fetches branches and commits and displays them.

Stimulus: GitHub API is unavailable or rate-limited.

Response: System displays an error message and allows retry; other modules remain usable.

4.27.3 Functional Requirements

REQ-GH-VIEW-01: The system shall fetch and display branch lists for the linked public repository in a read-only manner.

REQ-GH-VIEW-02: The system shall fetch and display commit history for the linked public repository in a read-only manner.

REQ-GH-VIEW-03: The system shall display an error message when GitHub API calls fail or are rate-limited.

REQ-GH-VIEW-04: The system shall allow retrying GitHub data retrieval later without breaking other system modules.

4.28 Gamification XP Rules and Anti-Cheating Controls

4.28.1 Description and Priority

The system awards XP after task completion and TA approval. XP values are set by the team based on difficulty/priority. Anti-cheating controls reduce abuse.

Priority: Medium

4.28.2 Stimulus/Response Sequences

Stimulus: Leader sets XP value for a task.

Response: System stores XP value for that task.

Stimulus: Task is finished and approved by TA.

Response: System awards XP once and updates dashboards.

Stimulus: User attempts to gain XP by toggling task status.

Response: System prevents XP award without TA approval and maintains history.

4.28.3 Functional Requirements

REQ-XP-01: The system shall allow the team leader to set an XP value per task based on difficulty and priority.

REQ-XP-02: The system shall award XP only after a task is finished and approved by the assigned TA.

REQ-XP-03: The system shall award XP only once per approved task completion.

REQ-XP-04: The system shall restrict XP edits after approval and record XP changes.

REQ-XP-05: The system shall reduce XP abuse by preventing XP gain through repeated status toggling without TA approval.

REQ-XP-06: The system shall keep a history of XP changes as part of anti-cheating controls.

4.29 Gamification Coins and Reward Store Items

4.29.1 Description and Priority

The system provides a reward store with items purchasable using coins. Store items include themes, avatar-related items, badges, and perks as described in supporting documents.

Priority: Medium

4.29.2 Stimulus/Response Sequences

Stimulus: User opens the reward store.

Response: System lists store items, including ownership status.

Stimulus: User purchases an item with coins.

Response: System validates balance, deducts coins, records purchase, and updates ownership.

Stimulus: User has insufficient coins.

Response: System blocks purchase and displays an error.

4.29.3 Functional Requirements

REQ-STORE-01: The system shall maintain a coin balance for each user as part of gamification data.

REQ-STORE-02: The system shall provide a reward store list of items including name, description, type, price in coins, rarity, and ownership status.

REQ-STORE-03: The system shall allow users to purchase a reward store item when sufficient coins are available.

REQ-STORE-04: The system shall reject purchases when the user's coin balance is insufficient and display an error.

REQ-STORE-05: The system shall deduct coins from the user balance upon successful purchase and record the purchase.

REQ-STORE-06: The system shall support store item types described in the documentation (e.g., theme, avatar-related items, badge, perk).

4.30 Leaderboard (Gamification Ranking)

4.30.1 Description and Priority

The system provides leaderboard data for gamification ranking as described in supporting backend documentation.

Priority: Medium

4.30.2 Stimulus/Response Sequences

Stimulus: User requests leaderboard view.

Response: System returns ranked users and includes current user rank context.

4.30.3 Functional Requirements

REQ-LB-01: The system shall provide leaderboard data including ranked users and relevant gamification fields (e.g., XP, coins, streak, achievements count) as described.

REQ-LB-02: The system shall include the current user's rank information in leaderboard responses where applicable.

REQ-LB-03: The system shall support leaderboard queries by type/period parameters where applicable as described.

4.31 Notifications (Email + In-app)

4.31.1 Description and Priority

The system sends notifications via email and displays them in an in-app notification bar for key events.

Priority: High

4.31.2 Stimulus/Response Sequences

Stimulus: A key event occurs (invite, request decision, meeting reminder, task approval).

Response: System creates an in-app notification and sends an email notification.

Stimulus: Email delivery fails.

Response: System preserves in-app notification and records email failure internally.

4.31.3 Functional Requirements

REQ-NOTIF-01: The system shall create an in-app notification when key events occur, including invites, request decisions, meeting reminders, and task approvals.

REQ-NOTIF-02: The system shall send email notifications for key events.

REQ-NOTIF-03: The system shall display in-app notifications immediately in a notifications bar/interface.

REQ-NOTIF-04: The system shall preserve in-app notifications even when email sending fails.

REQ-NOTIF-05: The system shall record email failure status internally when email sending fails.

4.32 AI Chatbot for SDLC Guidance

4.32.1 Description and Priority

The system provides an optional AI chatbot for SDLC/process guidance, using short text only, labeled outputs, and failure-safe behavior.

Priority: Medium

4.32.2 Stimulus/Response Sequences

Stimulus: User asks the AI chatbot a question.

Response: System shows AI notice, requests consent, sends short text to AI provider, and displays labeled response.

Stimulus: AI provider unavailable.

Response: System displays an error and continues without AI.

4.32.3 Functional Requirements

REQ-AI-CHAT-01: The system shall provide an AI chatbot for SDLC/process guidance.

REQ-AI-CHAT-02: The system shall send only short text inputs to the external AI service.

REQ-AI-CHAT-03: The system shall label AI chatbot outputs as AI-generated.

REQ-AI-CHAT-04: The system shall allow continued system usage when AI is unavailable.

4.33 AI Doctor/TA Recommendation

4.33.1 Description and Priority

The system may recommend doctors/TAs based on conditions, availability, team needs, and project idea summary. Recommendations are advisory only.

Priority: Low

4.33.2 Stimulus/Response Sequences

Stimulus: Leader requests recommendation.

Response: System obtains ranked suggestions with reasons and displays them; leader must choose manually.

4.33.3 Functional Requirements

REQ-AI-REC-01: The system shall provide advisory doctor/TA recommendations based on directory data and team needs as described.

REQ-AI-REC-02: The system shall present ranked suggestions with short reasons.

REQ-AI-REC-03: The system shall not auto-assign a doctor or TA based on AI recommendations.

REQ-AI-REC-04: The system shall preserve the normal request and accept/decline workflow after recommendations are shown.

4.34 AI Transparency + Block File Sending

4.34.1 Description and Priority

Before any AI action, the system must show a transparency notice and must block sending files to AI. AI remains optional.

Priority: High

4.34.2 Stimulus/Response Sequences

Stimulus: User initiates an AI feature.

Response: System shows notice, requires agreement, sends short text only if agreed, and blocks any file transmission.

4.34.3 Functional Requirements

REQ-AI-TR-01: The system shall display a notice stating that text will be sent to an external AI service before executing any AI action.

REQ-AI-TR-02: The system shall require user agreement before sending text to the external AI service.

REQ-AI-TR-03: The system shall send only short text to the AI service for AI-enabled actions.

REQ-AI-TR-04: The system shall block sending files or file content to the AI service.

REQ-AI-TR-05: The system shall allow users to continue using GPMS without AI features if they decline.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system shall meet measurable performance targets described in backend documentation, including:

NFR-PERF-01: The system shall achieve API response time < 200 ms at the 95th percentile for key endpoints under normal load.

NFR-PERF-02: The system shall deliver real-time messages (e.g., chat events) with < 50 ms delivery time under normal conditions.

NFR-PERF-03: The system shall achieve average database operation time < 100 ms for common queries under normal load.

NFR-PERF-04: The system shall complete search queries in < 300 ms under normal load.

NFR-PERF-05: The system should support typical faculty usage through pagination and efficient queries (consistent with NFR-10 scalability guidance).

5.2 Safety Requirements

While GPMS is not safety-critical in a physical sense, it shall protect academic operations and data integrity:

NFR-SAFE-01: The system shall support maintenance mode where user modifications are blocked and allowed roles may have read-only access as described.

NFR-SAFE-02: The system shall support backup operations and restoration procedures as described in administrator documentation.

NFR-SAFE-03: The system shall prevent unauthorized file access by enforcing role/team access controls for downloads.

NFR-SAFE-04: The system shall ensure that AI usage never transmits files and always requires user consent before transmitting text.

5.3 Security Requirements

Security requirements include those defined in the primary requirements source and expanded measurable controls described in backend/admin documentation:

NFR-SEC-01: The system shall use HTTPS/TLS for all traffic.

NFR-SEC-02: The system shall store passwords securely using hashed and salted storage, with no plain text passwords.

NFR-SEC-03: The system shall enforce permissions on backend APIs and not rely only on UI checks.

NFR-SEC-04: The system shall protect GitHub OAuth credentials/tokens using secure storage and minimal access.

NFR-SEC-05: The system shall implement rate limiting for authentication-related endpoints as specified in backend documentation (e.g., limiting password reset requests and controlling abusive request rates).

NFR-SEC-06: The system shall record audit logs for administrative and security-relevant actions as described in administrator documentation, including timestamp, user identity, action type, resource type, and IP address.

NFR-SEC-07: The system shall provide session management controls described in administrator documentation (e.g., session timeout and concurrent session limits), where configured.

NFR-SEC-08: The system shall implement role-based access control consistent with defined roles and permissions.

5.4 Software Quality Attributes

NFR-QUAL-01 (Usability): The system shall provide role-based dashboards and simple navigation so that core actions are performed in few steps.

NFR-QUAL-02 (Availability): The system should be available during academic working hours, with planned maintenance allowed.

NFR-QUAL-03 (Maintainability): The system shall follow a modular structure separating teams, requests, tasks, meetings, chat, files, admin, and AI modules.

NFR-QUAL-04 (Scalability): The system shall support many teams using pagination and efficient database queries.

NFR-QUAL-05 (Accessibility): The system shall support basic accessibility practices including keyboard navigation, readable text, and alternative text where applicable.

NFR-QUAL-06 (Testability): The project testing strategy shall target measurable automated test coverage goals described in backend documentation (e.g., >80% coverage goal).

5.5 Business Rules

Business rules stated in the primary requirements source include:

BR-01: A doctor can supervise many teams.

BR-02: A TA can support many teams.

BR-03: Each team must have exactly one doctor and exactly one TA to become active.

BR-04: Team size must be 3–8.

BR-05: No role overlaps.

BR-06: Students can belong to only one team in the cycle.

BR-07: Doctor and TA requests are separate; decline allows requesting another.

BR-08: Capacity is measured by number of assigned teams; requests are blocked when full.

BR-09: After the global deadline, teams cannot request; admin handles manually (extend deadline or assign).

BR-10: Tasks marked finished must be TA-approved to count as completed.

BR-11: Only active teams can use tasks/chat/meetings/files/repo modules.

BR-12: GitHub integration is public repositories only and read-only.

BR-13: AI is advisory only; no auto-assignment; send only short text and never files.

6. Other Requirements

- ✓ The system shall support sending verification emails, invites, request decisions, meeting reminders, and notifications via an email provider interface.
- ✓ The system shall support an external meeting link stored as a URL for meetings.
- ✓ The system shall integrate with GitHub OAuth and GitHub API to fetch read-only public repository branches and commits and handle API failures safely.
- ✓ The system shall implement AI features under strict transparency requirements, including explicit notice, optional use, and blocking file transmission.
- ✓ The system shall support data retention and archival considerations described as recommended defaults (graduation cycle plus 6–12 months retention) subject to policy finalization.
- ✓ The system validation checklist items defined in the requirements source shall be used for sign-off and acceptance review.

Appendix A: Glossary

GPMS: Graduation Project Management System.

Student: User role that registers and participates in a team.

Team Leader: A student with leader permissions who creates/manages team formation and requests.

Doctor / Supervisor: Faculty role responsible for supervising teams; accepts/declines doctor requests.

Teaching Assistant (TA): Support role; accepts/declines TA requests and approves finished tasks.

Administrator (Admin): Role with administrative access; creates staff accounts; configures deadlines; manual handling after deadline.

Join Code: Unique code generated for a team used for joining/inviting.

Public Team: Team visible in the public teams list for students without a team.

Private Team: Team not visible in the public teams list.

Active Team: Team status achieved only after both doctor and TA are assigned; unlocks execution modules.

Doctor Request: Request submitted by team leader to a doctor including project idea summary; has status (e.g., pending/accepted/declined).

TA Request: Request submitted by team leader to a TA including project idea summary; has status (e.g., pending/accepted/declined).

Project Idea Summary: Mandatory text submitted with doctor/TA requests describing the project idea.

Conditions: Doctor/TA-defined preferences/requirements visible in directory (text/tags).

Max Teams: Doctor/TA capacity limit used for request blocking.

Task: Unit of work within a team, with required fields and constrained status values.

Task Status Values: Allowed values: upcoming, in progress, finished.

Dependency: Relationship where one task depends on another task in the same team.

Circular Dependency: Invalid dependency cycle that must be detected and blocked.

TA Approval State: Approval status applied to finished tasks (Approved/Rejected).

Weekly Progress Draft: Auto-generated weekly summary based on tasks and approvals.

AI Weekly Rewrite: Optional AI feature that rewrites weekly draft text into clearer text, short text only.

Meeting: Scheduled team meeting with date/time and an external link.

Minutes: Stored meeting notes.

Topics: Items discussed in a meeting; can be assigned to members.

Team Group Chat: Single chat per active team including team members, assigned doctor, and assigned TA.

Quota: Enforced file limits (per file and per team).

OAuth: Authorization mechanism used for GitHub connection.

GitHub Public Repository: Repository accessible publicly; required for linking.

Commit / Branch: GitHub repository artifacts displayed read-only in GPMS.

XP: Experience points awarded for task completion under gamification rules.

Coins: Gamification currency used in reward store purchases.

Reward Store: Store of items purchasable with coins.

In-app Notification: Notification displayed within the application.

Email Notification: Notification delivered by email provider integration.

RBAC: Role-Based Access Control; enforcement of permissions by role/leader status.

Audit Log: Recorded history of security-relevant/admin actions including actor, action, time, and resource.

Appendix B: Analysis Models

This appendix references models and structures explicitly described in the uploaded sources.

1. System Architecture Model (PERN Architecture)

The backend documentation describes a PERN architecture with a Next.js/React frontend, Node.js/Express backend, and PostgreSQL database, with external integrations for email, AI provider, and GitHub.

2. Use Case Model (UC-01 to UC-24)

The requirements analysis enumerates core use cases including registration, verification, login, team creation/joining/recruitment, directory browsing, request workflows, tasks, approvals, weekly progress, meetings, chat, files, GitHub integration, notifications, and AI features.

3. Data Model / Key Entities (Minimum Set)

The requirements analysis defines a minimum key-entity list including User, StudentProfile, DoctorProfile/TAProfile, Team, DoctorRequest, TARequest, Task, WeeklyProgress, Meeting, ChatMessage, File, RepoLink, and XP/RewardHistory.

4. Workflow / State Models

Team Activation State: Team transitions to Active only when doctor and TA assignments exist.

Request State: Doctor/TA requests include a pending state and accept/decline decision outcomes.

Task State: Tasks constrained to upcoming/in progress/finished plus TA approval (approved/rejected) for finished tasks.

Weekly Progress Generation Flow: Weekly drafts generated from tasks and approvals, then reviewed and submitted.

Appendix C: To Be Determined (TBD)

The following items are unresolved, unclear, inconsistent across documents, or explicitly left open:

1. Verification token validity period (expiry duration is referenced by behavior but exact duration is not defined in the authoritative documents).

2. Join code expiration rules (expiration is implied in practice in guides, but specific duration/limits are not defined in the authoritative requirements list).

3. Exact task priority values (priority is required, but the enumeration/scale is not specified in the requirements analysis).

4. Exact definitions of "team privacy rules" for join behavior (FR-08 indicates behavior depends on privacy rules, but precise rules are not fully formalized).

5. Whether invitation status tracking is required or only optional (FR-09 mentions optional tracking/resend).

6. Exact mechanism and schedule for weekly progress auto-generation (the feature is required, but the triggering mechanism/schedule is not explicitly defined).

7. AI provider selection and operational limits (provider is external and unspecified; handling of quota exhaustion and request limits is not defined).

8. AI input/output maximum length constraints (documents describe "short text only," but numeric limits are not specified).

9. Coin earning rules (coins exist and are used for purchases, but the rules for awarding coins are not specified consistently as a single authoritative policy).

10. Long-term retention policy finalization (recommended retention is described as 6–12 months after the cycle, but faculty policy decision is explicitly deferred).

11. Final file upload configuration defaults (requirements specify 100MB/file and 500MB/team, while administrator/end-user guides describe smaller operational limits; the configured production defaults require confirmation).

12. Chat retention policy (guides mention chat usage but do not define whether chats are retained indefinitely, archived, or deleted after project completion).

13. Extent of notification channels (requirements mandate email + in-app; guides describe additional notification channels; final supported set requires consolidation).

14. Timeline view implementation details (simple timeline is required as a "should," but whether it is editable or auto-generated is not fully specified).

15. Scope alignment for additional features described in guides (some guide-described features exceed the explicitly listed FR set; final scope inclusion requires confirmation during approval review per the requirements review section).